

Linux

WPC

2026 年 4 月 4 日

目录

1 远程服务	3
1.1 NFS	3
1.2 SSH	3
2 内网穿透	4
3 远程桌面	6
4 环境	8
4.1 用户	8
4.2 软件包安装	8
4.3 环境变量	8
4.4 配置文件	8
5 存储设备	10
5.1 设备信息	10
5.2 文件系统	11
6 Shell 脚本	12
6.1 颜色	12
6.2 脚本格式	12
6.3 变量	12
6.4 vim	13
6.4.1 移动	13
6.4.2 删除	13
6.4.3 搜索	14

7 定时与服务	15
7.1 cron	15
7.2 systemd	15
7.3 GRUB	16
7.4 swap	17

1 远程服务

1.1 NFS

nfs service

```
sudo apt install nfs-kernel-server
vim /etc/exports
# 所有网段, ro只读, 同步内存和磁盘(async优先写内存), root用户映射为root用户
(root_squash映射为匿名用户)
/nfsroot *(rw, sync, no_subtree_check, root_squash)
sudo exportfs -a

sudo apt update
sudo apt install vsftpd
sudo systemctl start vsftpd
sudo systemctl enable vsftpd
sudo vim /etc/vsftpd.conf
anonymous_enable=YES
local_enable=YES //允许本地用户登录
write_enable=YES //允许用户写
sudo systemctl restart vsftpd.service
```

1.2 SSH

ssh service

```
install -dm700 ~/.ssh
scp ~/.ssh/id_ed25519.pub username@remoteip:
cat ~/id_ed25519.pub >> ~/.ssh/authorized_keys
rm ~/id_ed25519.pub
chmod 600 ~/.ssh/authorized_keys
```

2 内网穿透

Rathole Service

```
sudo vim /etc/systemd/system/ratholes.service
```

```
[Unit]
```

```
Description=Rathole Server Service
```

```
After=network.target
```

```
[Service]
```

```
Type=simple
```

```
Restart=on-failure
```

```
RestartSec=5s
```

```
LimitNOFILE=1048576
```

```
# with root
```

```
ExecStart=/usr/bin/rathole -s /etc/rathole/rathole.toml
```

```
# without root
```

```
# ExecStart=%h/.local/bin/rathole -s %h/.local/etc/rathole/rathole.toml
```

```
[Install]
```

```
WantedBy=multi-user.target
```

Rathole Toml

```
sudo mkdir -p /etc/rathole/rathole.toml

# server.toml
[server]
bind_addr = "0.0.0.0:2333" # `2333` 配置了服务端监听客户端连接的端口

[server.services.my_nas_ssh]
token = "use_a_secret_that_only_you_know" # 用于验证的 token
bind_addr = "0.0.0.0:5202" # `5202` 配置了将 `my_nas_ssh` 暴露给互联网的端口

sudo systemctl daemon-reload
sudo systemctl enable ratholes
sudo systemctl start ratholes

# client.toml
[client]
remote_addr = "myserver.com:2333" # 服务器的地址。端口必须
与 `server.bind_addr` 中的端口相同。
[client.services.my_nas_ssh]
token = "use_a_secret_that_only_you_know" # 必须与服务器相同以通过验证
local_addr = "127.0.0.1:22" # 需要被转发的服务的地址
```

3 远程桌面

relay

```
services:
  hbbs: # RustDesk ID/Rendezvous 服务器
    container_name: hbbs
    ports:
      - 21115:21115 # 用于 NAT 类型测试的 TCP
      - 21116:21116 # TCP打孔
      - 21116:21116/udp # UDP心跳/ID服务器
    image: rustdesk/rustdesk-server:latest
    command: hbbs
    volumes:
      - /data/rustdesk/hbbs:/root
    environment:
      - 'RELAY=192.168.143.6:21117' # 运行这些容器的服务器的【IP:port】或域名
      - 'ENCRYPTED_ONLY=1' # 开启加密
      - 'KEY=Key' # 自定义KEY, 去掉这一行可以自动生成
    depends_on:
      - hbbr
    restart: unless-stopped

  hbbr: # RustDesk 中继服务器
    container_name: hbbr
    ports:
      - 21117:21117 # TCP中继
    image: rustdesk/rustdesk-server:latest
    command: hbbr
    volumes:
      - /data/rustdesk/hbbr:/root
    restart: unless-stopped
```

server

```
# 走中继
sudo mkdir -p ~/.config/rustdesk
sudo tee ~/.config/rustdesk/RustDesk2.toml > /dev/null <<'EOF'
[options]
custom-rendezvous-server = "192.168.143.6"
relay-server = "192.168.143.6"
key = "Key"
direct-server = "Y"
direct-access-port = "21118"
EOF

rustdesk --get-id
rustdesk --password <your_password>
```

4 环境

4.1 用户

id 命令查看用户信息，用户定义在/etc/passwd 文件中，属组定义在/etc/group 文件中，一般来说，会为每个用户创建一个与用户同名的单成员属组。如果想切换用户可以使用 su 命令，可以使用 sudo 来以超级用户权限执行命令。编辑用户可以使用 useradd、userdel 和 usermod 命令。

4.2 软件包安装

在从源码安装软件时，通常需要先运行 ./configure。configure 是源码中提供的一个 Shell 脚本，它主要用于：分析和检测当前系统的构建环境、根据环境情况对源代码和编译选项进行适当调整、检查是否已安装编译所需的外部依赖和工具。在完成这些工作后，configure 会生成一个适合本机环境的 Makefile 文件。接着，就可以通过 make 命令来编译源码。编译后通常可以通过 make install 来安装软件，通常安装在/usr/local/bin 下。

4.3 环境变量

Shell 保存了两种数据类型：Shell 变量和环境变量。可以使用 printenv 命令查看环境变量、set 查看所有变量、alias 查看别名。

变量	说明
DISPLAY	屏幕名称，通常为:0，表示 X 服务器生成的第一个屏幕。
HOME	当前用户的主目录。
LANG	系统语言和区域设置。
OLDPWD	上一个工作目录。
PATH	由冒号分隔的目录列表，用于可执行文件的搜索。
PWD	当前工作目录。
SHELL	当前用户的默认 Shell。
TERM	终端类型。
USER	当前用户名。

表 4.1: 常见环境变量

4.4 配置文件

Shell 会话通常分为两类：登录会话和非登录会话，二者在启动时所读取的配置文件有所不同。当修改了配置文件后，可以通过 source 或 . 命令使改动立即生效。登录会话在启动时需要用户输入用户名和密码进行身份验证，而非登录会话则无需凭证即可进入。通常，通过图形界面（GUI）启动的终端属于非登录会话。

文件	说明
/etc/profile	应用于所有用户的全局配置。
~/.bash_profile	用户个人的登录 Shell 配置文件。
~/.bash_login	如果 ~/.bash_profile 不存在，则读取该文件。
~/.profile	如果前两者都不存在，则读取该文件。

表 4.2: 登录 Shell 读取的配置文件

文件	说明
/etc/bash.bashrc	应用于所有用户的全局配置。
~/.bashrc	用户个人的非登录 Shell 配置文件, 通常会在 ~/.bash_profile 中被调用。

表 4.3: 非登录 Shell 读取的配置文件

5 存储设备

5.1 设备信息

文件系统表 (file system tabl) /etc/fstab 列出会在系统引导时挂载的各种设备 (通常是硬盘分区)。

字段	说明
设备	文本标签或随机生成的通用识别码 (Universally Unique Identifier, UUID)。
挂载点	设备挂载到的目录。
文件系统类型	设备使用的文件系统类型, 如 ext4、swap。
挂载选项	挂载时的选项, 如 defaults、ro、noexec。
转储频率	用于 dump 命令的备份选项, 通常为 0 (不备份) 或 1。
自检顺序	用于 fsck 的文件系统检查顺序: 0=不检查, 1=先检查, 2=后检查。

表 5.1: /etc/fstab 文件字段

在 Linux 中, 可以使用 mount 命令挂载文件系统, 并通过 umount 命令卸载文件系统, 命令仅对当前会话有效, 如果需要开机自动挂载, 得在/etc/fstab 中写入内容。需要注意的是, 音频 CD 与常见的 CD-ROM 光盘不同, 音频 CD 本身不包含文件系统, 因此无法像普通数据光盘那样直接挂载。另外, 由于系统在读写设备时会使用缓冲区, 如果在数据尚未写回设备之前没有执行卸载操作, 直接移除或关闭设备, 就可能导致数据丢失或损坏。

设备	说明
/dev/lp*	并行端口打印机设备。
/dev/sd*	SCSI 磁盘及其分区。在现代 Linux 中, 内核将所有类似磁盘的设备 (如 SATA 硬盘、USB 闪存、数码相机等存储设备) 都视为 SCSI 设备。sda 表示第一个磁盘, sda1 是它的第一个分区, sdb 是第二个磁盘, 依此类推。
/dev/nvmeNn*	nvme 磁盘及其分区, N 是磁盘编号。
/dev/sr*	光驱设备 (CD-ROM/DVD)。
/dev/null	空设备, 写入数据会丢弃, 读取返回 EOF。

表 5.2: /dev 下常见设备及说明

5.2 文件系统

ISO Image

```
# 创建光盘映像文件 (ISO)
dd if=/dev/cdrom of=cd.iso          # (1)从已有 CD-ROM 光盘创建
genisoimage -o cd-rom.iso -R -J ~/cd-rom-files # (2)从目录生成 ISO 镜像

# 校验 ISO 文件完整性
md5sum /dev/cdrom
md5sum image.iso

# 刻录光盘
wodim dev=/dev/cdrw blank=fast # 擦除可重写光盘 (CD-RW / DVD-RW)
wodim dev=/dev/cdrw image.iso # 刻录 ISO 文件到光盘

# 挂载 ISO 镜像
mkdir /mnt/iso_image
mount -t iso9660 -o loop image.iso /mnt/iso_image
```

Disk Partitioning

```
lsblk      # 列出所有块设备
umount /dev/sdb1 # 操作设备前先卸载 (对分区)

# 分区 (对整个磁盘)
sudo parted /dev/sdb
mklabel gpt
mkpart primary ext4 0% 100%

sudo mkfs -t ext4 /dev/sdb1 # 格式化分区
sudo fsck /dev/sdb1        # 检查并修复文件系统错误
sudo blkid /dev/sdb1       # 查看分区UUID
sudo mount /dev/sdb1       # 挂载分区

# 将配置写入 /etc/fstab
UUID=<uuid> <mount_point> ext4 defaults 0 2
```

6 Shell 脚本

6.1 颜色

Shell 中也能显示颜色，分为文本颜色和背景颜色，关闭颜色的代码是\033[0m。

转义代码	文本颜色
\033[0;30m	黑色 (black)
\033[0;31m	红色 (red)
\033[0;32m	绿色 (green)
\033[0;33m	棕色 (brown)
\033[0;34m	蓝色 (blue)
\033[0;35m	紫色 (magenta)
\033[0;36m	青色 (cyan)
\033[0;37m	浅灰色 (light grey)

表 6.1: 常用文本颜色代码

转义代码	文本颜色
\033[1;30m	深灰色 (deep grey)
\033[1;31m	浅红色 (light red)
\033[1;32m	浅绿色 (light green)
\033[1;33m	黄色 (yellow)
\033[1;34m	浅蓝色 (light blue)
\033[1;35m	浅紫色 (light magenta)
\033[1;36m	浅青色 (light cyan)
\033[1;37m	白色 (white)

表 6.2: 高亮文本颜色代码

转义代码	背景颜色
\033[40m	黑色
\033[41m	红色
\033[42m	绿色
\033[43m	棕色
\033[44m	蓝色
\033[45m	紫色
\033[46m	青色
\033[47m	浅灰色

表 6.3: 背景颜色代码

6.2 脚本格式

在 Shell 脚本 的第一行通常会写上#!/bin/bash，这行语句称为 shebang，告诉操作系统在执行脚本时，应该调用哪一个解释器。

6.3 变量

在 Shell 中，变量可以像 Python 一样直接赋值，不过这种方式定义的变量都是 全局变量；如果需要定义局部变量，则必须使用 local 关键字。除了直接赋值，还可以通过 read 从标准输入读取

用户输入并赋值，一般配合 `echo -n` 给出提示，也可以通过 `$0`、`$1` 等读取命令行参数，`##` 表示参数的个数。

在需要向命令传递输入时，Shell 还提供了两种重定向方式：`here string` 和 `here document`。Here String 使用 `<<<`，可以把一行字符串作为标准输入传递给命令；Here Document 则适合多行输入，它把分隔符之间的所有文本传递给命令的标准输入。

其中 EOF 是分隔符，可以自定义，只要首尾一致即可。

```
cat <<EOF
这是第一行
这是第二行
EOF
```

6.4 vim

如果你在 vi 中不清楚自己处于什么模式，可以通过两次 `esc` 来返回命令模式。

6.4.1 移动

命令	说明
0	移动到行首
^	移动到行首第一个非空白字符
\$	移动到行尾
2G	移动到第二行
G	移动到最后一行
gg	移动到第一行

表 6.4: vim 常用移动键位

6.4.2 删除

`d` 删除其实是剪切操作，`y` 是复制，用法和 `d` 类似，`p` 是粘贴命令。

命令	说明
x	删除当前字符
2x	删除当前 2 个字符
dd	删除当前行
2dd	删除当前两行
dG	从当前行删到最后一行
d20G	从当前行删到 20 行
d\$	从当前字符删到行尾

表 6.5: vim 常用删除键位

6.4.3 搜索

- 行内搜索：fa 就是移动到行内下一个 a 所在之处，使用; 能继续搜索。
- 全局搜索：/进行全局搜索，使用 n 继续搜索。
- 行内替换：:s/Line/line/g, s 是替换命令，g 代表找到的结果全部替换。
- 全局替换：命令为%s/Line/line/g, %代表搜索整个文件，也可以写成1,\$，代表从第 1 行到最后一行。

7 定时与服务

7.1 cron

通过运行 `crontab -e` 命令来编辑定时任务，文件中每一行表示一个定时任务，格式如下：

cron 任务格式

```
* * * * * /usr/bin/python3 example.py >> output.log 2>&1
| | | | |
| | | | +---- 星期 (0 - 7) (0 和 7 都是星期天)
| | | +----- 月 (1 - 12)
| | +----- 日 (1 - 31)
| +----- 小时 (0 - 23)
+----- 分钟 (0 - 59)
```

时间表达式	含义
0 * * * *	每小时的第 0 分钟执行
0 9 * * *	每天上午 9 点执行
0 9 * * 1-5	每周一到周五上午 9 点执行
*/10 * * * *	每 10 分钟执行一次

表 7.1: 常见的 Cron 时间表达式说明

注意：编辑保存后无需重启 cron 服务，它会立即生效。如果系统宕机，错过的任务不会被补执行。

7.2 systemd

systemd 是现代 Linux 发行版中常用的初始化系统和服务管理器，用于启动和管理系统服务及进程。systemd 使用单个二进制文件 `/usr/bin/systemd` 作为初始化进程 (PID 1)，并通过一系列配置文件来定义和管理服务。这些配置文件通常位于 `/etc/systemd/system/` 目录下，文件扩展名为 `.service`。

```
/etc/systemd/system/rathole.service
```

```
[Unit]
Description=Rathole Server Service
After=network.target

[Service]
Type=simple
Restart=always
RestartSec=5s

ExecStart=/usr/bin/rathole -s /etc/rathole/rathole.toml

[Install]
WantedBy=multi-user.target
```

cron 任务格式

```
sudo systemctl daemon-reload
sudo systemctl start rathole
sudo systemctl enable rathole
sudo systemctl status rathole
```

7.3 GRUB

GRUB (GRand Unified Bootloader) 是一个引导加载器程序，负责在计算机启动时加载和引导操作系统内核。GRUB 默认配置文件是 `/etc/default/grub`，`/etc/default/grub.d` 目录里面也存放着一些配置。使用 `update-grub` 命令时，会用上述配置生成最终配置文件 `/boot/grub/grub.cfg`。

`uname -r` 查看当前使用内核，要列出当前系统中已安装的 Linux 内核版本，可以使用命令 `dpkg --get-architecture | grep linux-image`。

命令输出解释：

- `ii`：表示已安装，并成功配置。
- `rc`：表示已删除，但配置文件仍然存在。

如果你想删除旧内核版本，可以使用命令 `sudo apt purge linux-image-x.x.x-generic`。删除内核后，需要运行 `sudo update-grub` 命令来更新 GRUB 配置。如果你需要清理残留依赖包，运行 `sudo apt autoremove`。

进入 grub:

- 命令行: 开机长按 `esc` 进入命令行。
- 图形界面: 开机长按 `Shift`, 如果不行, 先进入命令行, 运行 `normal` 命令, 之后按一下 `esc`。

7.4 swap

systemd 服务配置文件示例

```
free -h
swapon --show
sudo fallocate -l 1G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```